

Developer experience report — SecondWave

Track 2 — Lending · XRPL public Devnet · September 2026

Who this is for: the judges and the Ripple team. **What it is:** what we actually went through building a product on the XRPL lending protocol — not a catalogue of error codes.

Team: Hugo (vaults, loans, resale) · Noé (risk analyst, wallet) **Track:** 2 — Lending · private vaults + on-chain credentials **Network:** XRPL public Devnet · September 2026

Technical evidence (scripts, hashes, versions) is in the appendix. Here we describe what blocks a human.

In thirty seconds

1. **A lender’s “collateral” is almost never what you think it is.** A default can push **95 % of the loss onto depositors**, even when the broker shows a large reserve.
2. **While the money is locked in, the displayed price can stay beautiful.** A late loan changes nothing on screen until someone formally records the problem. We found **22 vaults in exactly that state** among the 40 largest on the network, without having created a single one.
3. **“Success” does not mean the trade happened.** Delivery and no-op are indistinguishable in the response, and three send modes out of four hand over the shares without collecting the price.
4. **A signed offer did not survive waiting** — 72 seconds, and invalidated the moment either party did anything else. We found how to make it durable; it is documented nowhere.
5. **Nothing in the wallet ecosystem can sign a batched exchange**, so any peer-to-peer market is forced to be custodial.

What we built

SecondWave is an **exit market** for closed-ended vault shares, with a **risk rating** next to every offer.

The idea is simple. Once money is deposited in a fixed-term vault, it cannot be withdrawn until the redemption date. During that time, the loans inside can go bad. We let you **resell your share** to someone else, and we say whether the discount is an **opportunity** (someone needs cash) or a **trap** (the vault is already sick).

Getting there meant running the whole cycle on Devnet: credentials, vaults, deposits, brokers, loans, defaults, then resale. It is that cycle — not an isolated tutorial — that surfaced the frictions below.

1. “Collateral” is not what you think

This is **the** point of the project. Hugo measured it at the protocol level; Noé put it at the heart of the analyst.

We are told about **first-loss capital**: a reserve the broker deposits, which should be eaten **before** the savers’ money. The name — and often the screen — suggests: “if they have 5,000 in the pot, that 5,000 protects depositors.”

That is not what the network does.

At default, the protocol takes **only a small computed slice** — a percentage of a percentage. The rest of the pot **stays with the broker**. Depositors pay the difference. And they cannot leave the vault while it happens.

Same numbers, two readings:

What you see	What actually happens
Reserve shown: 5,000	Only 500 can be taken at default
“We’re well covered”	95 % of the loss goes to depositors
The surplus looks reassuring	The remaining 4,500 never move

The official documentation example **already computes** this kind of result (about 11 of protection against a 1,090 default). It never says it in one sentence: *almost none of that default is first loss*.

Two corollaries, just as simple:

- **As soon as no loan is outstanding**, the broker can **pull back 100 %** of the reserve. It is not a permanent cushion.
- **A late loan does not lower the vault’s price.** The displayed figure only moves when someone formally declares a loss. Until then, the depositor is stuck behind a price that is too good.

What we’re asking for: one sentence, clearly visible, on the “First-Loss Capital” page: *the real protection at default is not the money in the pot, it is the small cap set by the rates*. And, later, a pre-computed field (“maximum protection”) so wallets don’t have to reinvent it.

2. Once you're in, you don't get out — sometimes not at all

A fixed-term vault **forbids withdrawal** for the whole investment period. That is the product. We accepted it.

The trap is the **combination** with on-chain identity. Our vaults are private: only holders of an accepted credential can enter, and also **receive** a share.

If that credential expires or is revoked while the money is locked in:

- you **cannot withdraw** (too early);
- you **cannot sell** to an otherwise valid buyer (the network refuses the transfer).

The person who most needs an emergency exit is the one denied it. Withdrawal remains possible **at the redemption date**, even without a credential — but until then the position is frozen.

What we're asking for: decide clearly whether selling *outward* (leaving the vault) should stay possible when you are no longer a member. Today the safety net arrives too late.

3. The official wallet cannot deposit XRP

Noé tried to subscribe from the **Ripple wallet extension**, as a normal user would, not as a script.

The vault expects an XRP amount in a plain form. The extension always sends the amount **as if it were a token** (an object with a currency). The network refuses. The message only says the field is invalid — **without saying the type is wrong**. The user **cannot edit** the JSON.

Result: **dead end**. You cannot join an XRP vault from the official tool.

We fixed the bug in a copy of the extension and opened a pull request. SecondWave itself is not at fault here: this is a depositor's first real contact with XLS-65.

What we're asking for: if the vault is in XRP, send the amount as a plain string; keep the “token” object for IOUs only. And a message that says *“this vault expects XRP, not a token”*.

4. “Success” does not mean the trade happened

This is the friction that cost us the most code, and the one we did not see coming. It concerns **XLS-56 (Batch)**, the transaction our whole settlement rests on.

To resell a share we send **several operations at once**, all or nothing: the buyer authorises themselves to receive the shares, the shares move, the money moves. Three legs, one transaction.

The network answers **success**. The problem is that this word does not separate two opposite situations.

What the server returns when the trade happened, and when it did nothing:

Field	Trade succeeded	Nothing applied
Response	<code>accepted</code>	<code>accepted</code>
Applied	<code>true</code>	<code>true</code>
Code	<code>tesSUCCESS</code>	<code>tesSUCCESS</code>
Text	“The transaction was applied”	“The transaction was applied”
Metadata	no value movement	no value movement

Eight sends replayed: five delivered, three did nothing. **One single observed combination**. Nothing in the response separates them.

We froze the demonstration on-chain so it can be checked without us. Two Devnet transactions, openable in the explorer: one delivered 3 XRP, the other did nothing. Here is everything the ledger records about each:

	A — delivered	E — did nothing
Code	<code>tesSUCCESS</code>	<code>tesSUCCESS</code>
Metadata nodes	Deleted Ticket, Modified AccountRoot, Modified DirectoryNode	Deleted Ticket, Modified AccountRoot, Modified DirectoryNode
Balance change	−150 drops (the fee)	−150 drops (the fee)

Rigorously identical. Neither leaves, in its own metadata, the faintest trace of what moved — or didn't. The links are in probes-marche/PREUVES.md.

We are not saying the code is wrong. “Nothing” is a legitimate outcome of all-or-nothing. We are saying you cannot tell which case you are in. XLS-56 does specify a per-leg execution report, **exactly for this**. It is absent from what the server returns.

Three consequences we had to pay for in code:

- **Rebuild the legs by hand.** They exist in the ledger, as separate transactions in the same block, with their own hashes. You have to go and fetch them account by account over the block's range. It is the most fragile part of our settlement, and it only reconstructs what the server already knows.
- **Compare balances before and after.** That is our only proof a trade happened. Any serious product will have to do the same.
- **Never pre-check.** The simulation tool refuses batched sends — it answers “not implemented”. The transaction everything depends on is the only one you cannot dry-run.

And the choice of mode is a silent trap. We replayed the same exchange under all four send modes. **Three out of four hand over the shares without collecting the price**, while announcing success. You cannot forget to pick a mode — but you can pick the wrong one, and nothing flags it. For value-against-shares, only one is correct. Two details from the same family, found by typing next to the target:

- a batched send with **fewer than two legs** is refused with “array is empty”, when the array is not;
- the buyer's authorisation leg **must come first**. Placed anywhere else, the share delivery fails and everything is rolled back. Nothing says so.

What we're asking for:

1. That the response carry **the index of the failing leg and its code**. One integer and one code — the field is already specified, it just needs filling.
 2. A warning at the top of the page: *to exchange value, use all-or-nothing only*, and a helper in the SDKs that enforces it.
 3. That the simulation tool accept batched sends. It is the transaction we most need to test.
-

5. Error messages don't say what to fix

This was the running theme of the whole weekend. A few scenes from real life:

“Malformed transaction.” A vault refused because the dates are too close, too far apart, because a field is forbidden on XRP, or because a flag is missing. The server says the same sentence every time. The useful messages exist only **inside the JavaScript SDK**. Anyone signing another way gets nothing.

“Expired.” That single word means four different things: a repayment slightly too late, a deposit outside the entry window, a loan started too late, an expired credential. Impossible to tell which without re-reading everything by hand. And for the late repayment, the network expects a **special flag** the message never mentions.

“No permission.” Often it is not about rights at all. It is “your loan dates are too close to the vault's redemption” or “this vault type does not accept a broker”. You go looking for an identity problem; it is a calendar problem.

An invisible rule, a lost session. A loan's grace period must last **at least one minute**, and **no longer** than the interval between two payments. Nowhere in the docs. The refusal is, again, “malformed”. We tested three false leads before sweeping everything.

Same family: three fields are all called “rate” and do not mean the same thing (annual interest, reserve floor, share liquidated at default). Without a table you get the display wrong by a factor of 10 or 20.

What we're asking for: name **the field** and **the rule** in the message. **expired** and **no permission** should become sentences like “*late repayment: add the corresponding flag*” or “*last payment too close to the redemption date*”.

6. You can't simply ask “show me this loan”

There is a read method for **the vault**. There is **none** for a loan, for a broker, or for the list of share holders.

To analyse a vault we have to:

1. read the vault;
2. list the objects of the vault's pseudo-account (the brokers);
3. for each broker, list its loans again;
4. for share holders, **replay the entire history** of the account, because nobody provides the list.

A loan repaid and deleted **disappears**. There is no on-chain credit history. Any serious risk rating requires an indexer alongside.

You also cannot **simulate** a loan before sending it (the second signature is missing), nor simulate a batched exchange. The hardest operations to build are the ones you cannot dry-run.

What we're asking for: three business reads — the loan, the broker, the share holders — and the right to simulate a loan and a batched exchange.

7. What runs on the network isn't in the docs

Devnet already runs a **newer version** of the protocol (three-phase vaults: subscription, investment, redemption). The fields exist. The SDK knows them. **No published page describes them.**

We learned everything by trial and error:

- a broker **only exists** on a fixed-term vault;
- a “start date” field is **forbidden** at loan creation (it is computed automatically);
- a “free data” field is **accepted then dropped** — apparent success, information lost;
- an XRP vault **refuses** a scale field that only exists for tokens.

Same surprise on the resale side: **escrowing shares works**, up to an atomic swap without any batched operation. Nothing says so. We found it by typing next to the target.

Conversely, the native order book still **refuses** to quote these shares (“this logic is disabled”). The code is there, the switch is off. We don't know whether that's for tomorrow or for two years from now — and it decides whether our peer-to-peer market still makes sense.

8. A signed offer does not survive waiting

This is the problem that decides whether you build a **market** or a **handshake**. We only understood it by trying to make an offer wait.

When two people swap shares against money, each one signs. But what each one signs contains **the other's transaction counter**. As a result, as soon as either party does any other operation, the already-signed offer becomes invalid. And the envelope carries a short deadline — we measured **72 seconds** on Devnet.

Concretely: the seller cannot sign now and let the buyer show up later. Both must be at their screens, at the same time, for about a minute. That is not an order book, it is an appointment.

What we found, and it changes everything. The protocol can reserve a transaction number in advance. Using it instead of the ordinary counter unlocks the whole thing. We measured seven cases:

What we tested	Result
Reserved number on the envelope	accepted
Reserved number on each inner leg	accepted
Offer signed, then both accounts do other things, then we send it	it goes through
Deadline pushed to 10 min, 1 h, 24 h, 7 days, 8 years	all accepted
No deadline at all	accepted

A signed offer therefore becomes **durable**. And cancellation exists too: consuming the reserved number makes the offer permanently unexecutable, **even if the other party has already signed it**. Measured cost: **1 drop**.

Two remarks that matter for what follows:

- **None of this is written down.** The JavaScript SDK already reads the reserved number when it assembles signatures — the mechanism is anticipated. No page says a batched exchange can be carried that way, nor what it enables.
- **An offer with no deadline is not desirable either.** As long as the buyer has signed and the seller has not, the seller holds a **free option**: they execute on the day that suits them, possibly a year later, on a vault that will have changed nature in the meantime. The buyer would pay yesterday's price for today's asset. So we put a deadline back — but on the buyer's **commitment**, not on the seller's listing, which commits nobody.

What we're asking for: document that a batched exchange can be carried by a reserved transaction number, and what that implies — it is the difference between a market and an appointment. And, on the same page, say that having no deadline transfers a free option to whoever signs last.

9. You can't ask for the list of vaults

To rate a vault's risk, you first have to find it. There is **no read method** that answers “here are the vaults”. The only route is to **sweep the ledger** filtering by object type.

Measured on Devnet: **489 vaults in 11 seconds**, across 60 pages — and the sweep was still **truncated**. Then, knowing whether any of them deserves a look means re-reading each one in full: for the 40 largest, about 27 seconds more.

This is not a performance detail. It decides whether an independent analyst can exist at all. And it is worth the trouble: among those 40 vaults, **22 had an undeclared defaulted loan**. The signal we were looking for does exist in the wild, on addresses that are not ours — but nobody can see it without sweeping the whole chain.

Same family as point 6: the loan, the broker, the share holders are missing... and so is the list of vaults itself.

What we're asking for: a read method that lists vaults, with at minimum a filter by owner and by type. Failing that, say clearly in the docs that any product in this space needs an indexer, and what it should look like.

10. No wallet can sign a batched exchange

This is our last point, and possibly the most damaging one for the ecosystem.

The batched exchange works on-chain. We used it dozens of times. But it is **out of reach of a wallet**. We read the source of the official developer wallet: **the word appears nowhere**. Neither batched exchange nor escrow. It can create and send tokens, accept a credential, and it does have a vault deposit screen — the one from point 3, which does not work in XRP. But it can sign **neither** of the two operations that enable an atomic swap.

The connection protocol between a site and a wallet does declare a “sign on behalf of another account” method, which is exactly what would be needed. No wallet implements it for batched exchanges.

The consequence is brutal for a product: **any peer-to-peer market has to sign server-side**, with its users' keys. In other words, become custodial — precisely what this protocol is trying to avoid. Our interface does it, locally, and we own that as a demo limitation. We would not put it online.

What we're asking for: bring the batched exchange into the official wallet and into the connection standard, with a readable review screen (“you deliver X, you receive Y, all or nothing”). Until then, the most interesting part of the protocol stays reserved for people who write scripts.

11. A proposal: escrow without a named destination

The previous points converge on the same wall, and we would like to propose a way out.

Today, **the seller must know their buyer before committing**. This is true on both rails:

- the batched exchange: the buyer's address is **inside** what the seller signs;
- escrow: it requires a **named destination** at creation.

So there is no way to say “I sell 1,000 shares at this price, to whoever takes them first”. The native order book still refuses to quote these shares. A seller who wants out must **find their buyer elsewhere**, then come back and sign. That is why our order book is off-chain — not an architectural choice, an unavailability.

****Why we think a bearer escrow would be reasonable *here*.**

The usual objection is obvious: an escrow anyone can claim is an open door. But in a private vault, **“anyone” does not exist**. Access is already bounded by a permissioned domain (XLS-0080) and the credentials it accepts (XLS-0070). Only domain members can hold these shares — the network checks it at deposit, at transfer, and **again when the escrow is finished**, which we tested.

In other words, the control a named destination provides is **already provided by the domain**. Naming it a second time adds no security: it only removes the possibility of a market.

What we're asking for: the ability to create a share escrow pointing at **a domain** instead of a destination. The first member in good standing to finish takes the position, against the agreed price. The seller commits once, publicly, without knowing who will answer — and without any loss of trust, since every possible responder is already authenticated.

That would give the protocol a real order book for private vaults, without waiting for the native book to accept these shares — and without anyone having to trust an off-chain intermediary like ours.

What reassured us

Not everything is broken. These held, and they deserve to be kept.

- **The vault owner cannot push back the redemption date.** Once written, it is written. No “six more months” decided after the fact.
 - **Losing your credential does not prevent withdrawal** at the scheduled date. Only a token freeze (reversible) blocks an exit.
 - **Vault money does not leak** into ordinary payments. You also cannot nest a vault inside another vault.
 - **Not one drop lost to rounding** over hundreds of round trips, in XRP as in tokens.
 - **All-or-nothing really holds.** We could not break it. If the buyer signs and the seller then changes the price, the network refuses.
 - **The identity gate is watertight** and re-checked everywhere: deposit, transfer, escrow, including at escrow finish.
-

What we're asking for, in seven points

1. **Tell the truth about collateral**, in one sentence, where people read it. The pot is not the protection.

2. **Error messages that name the field and the rule.** Fewer catch-all “malformed”, “expired”, “no permission”.
3. **Be able to read a loan, a broker, who holds the shares — and the list of vaults** — without rebuilding the world by hand.
4. **Make “success” mean “the trade happened”:** the index of the failing leg and its code, and a warning about the choice of send mode.
5. **Document what already runs** (three-phase vaults, date rules, the real meaning of the rates, share escrow) — and say whether the native book will ever quote these shares.
6. **Document that a batched exchange can be carried by a reserved transaction number.** That is what separates a market from a 72-second appointment, and it is written nowhere.
7. **Put the batched exchange into wallets.** As long as none can sign it, every peer-to-peer market is forced to be custodial.

And a proposal rather than a request: **an escrow addressed to a domain** rather than to a person (point 11). The official wallet should also be able to deposit XRP into a vault (point 3).

Questions for Ripple

1. **The native order book** — will it ever quote vault shares, and will it respect private vaults? If yes, our peer-to-peer market becomes a temporary bridge. If no, it stays the product.
2. **The ban on exiting during investment** — is it final, including for someone who is no longer a member?
3. **Reselling through a batched operation** — is that a supported use, or a side effect you may break later?
4. **An over-prudent broker** (more money in the pot than the cap) — will they ever be able to offer genuinely more protection, without gaming the rates?
5. **Carrying a batched operation on a reserved transaction number** — is that intended, tolerated, or accidental? The SDK already does half of it; we built on top and we would like to know whether we were right.
6. **An escrow addressed to a domain** rather than to a person — does it collide with something we cannot see? It is our only real lead towards an on-chain order book for private shares.

Where the proofs are

This text merges the work of both sides of the team. The long versions remain the reference for a mentor or a reproduction.

File	Who	What
FEEDBACK.md	mostly Hugo	Technical report (SDK, batched operations, messages, missing reads)
FEEDBACK_XLS65_XLS66.md	mostly Noé	First-loss + wallet note, with formulas and proofs
probes/FRICTIONS.md	Hugo	Vault / loan reproductions
probes-marche/FRICTIONS.md	Hugo	Resale / escrow reproductions
FRICTIONS-annexe.md	Hugo	Long version, market side
probes-marche/13-tickets-et-batch.mjs	Hugo	Durable offer: 7 cases, legs verified in the ledger (point 8)
probes-marche/14-annulation-par-ticket.mjs	Hugo	Enforceable cancellation of an already-signed offer (point 8)
probes-marche/15-fenetre-maximale.mjs	Hugo	How far the deadline can be pushed: 10 min → 8 years (point 8)
probes-marche/16-deux-achats-en-parallele.mjs	Hugo	Two simultaneous purchases, and a no-funds send that answers “success” (point 4)
probes-marche/12-vaults-publics.mjs	Hugo	Network sweep: 489 vaults, 22 in undeclared default (point 9)
probes-marche/PREUVES.md	Hugo	8 Devnet transactions openable in the explorer — points 4 and 8

About 450 cases replayed on Devnet, around thirty vaults created and 489 swept. Every technical point has a script in the repository.

Security leads are passed on verbally, as the rules require. They are not in this document.